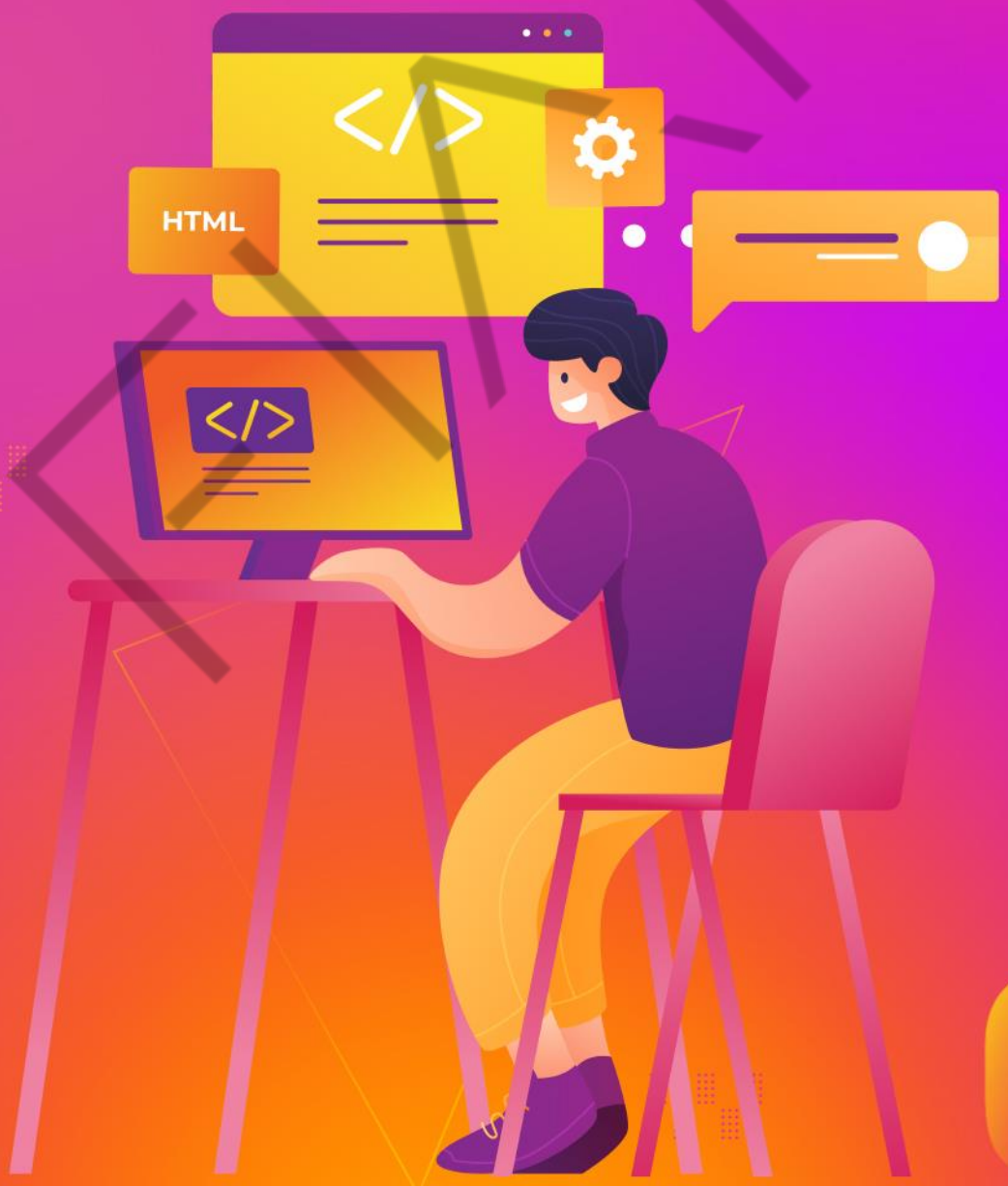
FIAP ON

DESENVOLVIMENTO
JAVA ENTERPRISE

EJB – ENTERPRISE JAVABEANS

JOSÉ YOSHIRIRO



6

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 6.1 – Funcionamento dos EJBs no AS.....7
Figura 6.2 – Funcionamento de um Stateful Bean numa sessão do cliente.....9
Figura 6.3 – Funcionamento de um Stateless Bean numa sessão de cliente 10
Figura 6.4 – Funcionamento de um Singleton Bean 11
Figura 6.5 – Componente *JBoss AS, Wildfly & EAP Server Tools* do *JBoss Tools*...18
Figura 6.6 – Componente *Maven Integration for Eclipse APT* do *JBoss Tools* 19
Figura 6.7 – Página de download do *WildFly* 20
Figura 6.8 – Solicitando a instalação de um novo servidor no *Eclipse* 20
Figura 6.9 – Instalação de um *WildFly* no *Eclipse* 21
Figura 6.10 – Tela de indicação do caminho do *WildFly* no *Eclipse*.....21
Figura 6.11 – *WildFly 10.x* na aba *Servers* 22
Figura 6.12 – Saída gerada no *Console* após a inicialização do *WildFly* 22
Figura 6.13 – Página inicial padrão do *WildFly* 23
Figura 6.14 – Indicação do *archetype* do tipo *maven-archetype-webapp* no *Eclipse* 24
Figura 6.15 – Projeto *smartcities-ejb* no *WildFly 10.x* 25
Figura 6.16 – Página do *smartcities-ejb* no navegador 25
Figura 6.17 – Página do *ContadorServlet* no navegador 39
Figura 6.18 – Página do *EstabelecimentosServlet* no navegador 41

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE TABELAS

Tabela 6.1 – Configurações de agendamento para a anotação "@javax.ejb.Schedule" 15

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 6.1 – Exemplo de Session Bean Stateful 8
Código-fonte 6.2 – Exemplo de Session Bean Stateless 9
Código-fonte 6.3 – Exemplo de Session Bean Singleton 10
Código-fonte 6.4 – Exemplo de Bean com todas as chamadas assíncronas 12
Código-fonte 6.5 – Exemplo de Bean com método assíncrono 13
Código-fonte 6.6 – Exemplo de Bean com método agendado 14
Código-fonte 6.7 – Exemplo de Bean com método agendado desde o início da execução do projeto 14
Código-fonte 6.8 – Exemplo de Bean 17
Código-fonte 6.9 – Configuração de *build* do projeto *smartcities-ejb* 26
Código-fonte 6.10 – Dependências projeto *smartcities-ejb* 27
Código-fonte 6.11 – Arquivo *web.xml* configurado para *Servlets* 3.1 28
Código-fonte 6.12 – Exemplo de arquivo *module.xml* para Oracle 29
Código-fonte 6.13 – Exemplo de arquivo module.xml para *MySQL* 29
Código-fonte 6.14 – Exemplo de *Data Source* para Oracle no *standard.xml* do *WildFly* 30
Código-fonte 6.15 – Exemplo de *Data Source* para *MySQL* no *standard.xml* do *WildFly* 30
Código-fonte 6.16 – Arquivo *persistence.xml* do projeto *smartcities-ejb* 31
Código-fonte 6.17 – Código do ContadorPesquisasService 32
Código-fonte 6.18 – Código do HistoricoPesquisasService 33
Código-fonte 6.19 – Código do EstabelecimentosService 35
Código-fonte 6.20 – Código do SmartcitiesListener 36
Código-fonte 6.21 – Código do ContadorServlet 38
Código-fonte 6.22 – Código do *contador.jsp* 38
Código-fonte 6.23 – Código do *EstabelecimentosServlet* 41
Código-fonte 6.24 – Código do *estabelecimentos.jsp* 41
Código-fonte 6.25 – Código da interface EstabelecimentosServiceRemote 42
Código-fonte 6.26 – Interface EstabelecimentosServiceRemote indicada como “versão remota” 42
Código-fonte 6.27 – Código da interface EstabelecimentosServiceLocal 43
Código-fonte 6.28 – Interface EstabelecimentosServiceLocal indicada como “versão local” 43
Código-fonte 6.29 – Dependências necessárias para criar um cliente EJB 44
Código-fonte 6.30 – Classe Java que funciona como Cliente EJB 45

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

6 EJB – ENTERPRISE JAVABEANS.....6
6.1 O que são EJBs? 6
6.1.1 Session Beans 7
6.1.2 Chamadas assíncronas.....11
6.1.3 Métodos agendados (*Schedule*).....13
6.2 Projeto EJB + JPA +JSP + Servlets 17
6.2.1 Instalando o *JBoss Tools* no *Eclipse*.....18
6.2.2 Instalando o AS *WildFly* 19
6.2.3 Criando um projeto EJB com Maven.....23
6.2.3.1 Alterando o pom.xml 26
6.2.3.2 Alterando o web.xml 27
6.2.3.3 Reaproveitando (copiando) classes do projeto JPA 28
6.2.3.4 Criando um *Data Source* no *WildFly* 28
6.2.3.5. Criando o persistence.xml 31
6.2.3.6 *Singleton Bean*: ContadorPesquisasService. 32
6.2.3.7 *Stateful Bean*: HistoricoPesquisasService. 33
6.2.3.8 *Stateless Bean*: EstabelecimentosService. 34
6.2.3.9 *Serlvet Listener*: SmartcitiesListener. 35
6.2.3.10 ContadorServlet e contador.Jsp 37
6.2.3.11 EstabelecimentosServlet e estabelecimentos.jsp 39
6.3 Interfaces locais e remotas de um *Session Bean*.....42
6.3.1 Expondo o EstabelecimentosService para acesso remoto.....42
6.3.2 Expondo o EstabelecimentosService para acesso local 43
6.3.3 Cliente para o projeto EJB.....44
REFERÊNCIAS.....47

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6 EJB – ENTERPRISE JAVABEANS

6.1 O que são EJBs?

EJB (Enterprise JavaBeans, ou “Componentes Java Corporativos”, em tradução livre) é um conjunto de tecnologias indicadas para aplicações de médio e grande porte, daí o nome *Enterprise* (ou “Corporativos”).

Para que um projeto Java use EJB, ele precisa ser do tipo EJB e estar instalado em um ***Jave EE Server*** ou ***Application Server*** **JEE** (ou “Servidor de Aplicação JEE”, em tradução livre), ou simplesmente **AS**. Ou seja, não se executa uma classe com método **main()** para usar EJB. Os principais *AS* do mercado são **WildFly** (livre e de código aberto), **RedHat JBoss** (versões livre e paga), **IBM WebSphere** e **Oracle WebLogic**.

Um AS é uma aplicação Java bem robusta, que levanta nele diferentes ***Containers*** (serviços que ele é capaz de executar). Existem vários tipos de *Containers*: para o envio de e-mails, para o gerenciamento de autenticação e autorização de usuários, para a execução de *JSP* e *Servlets* (que será estudado mais adiante), para a execução de processamentos em lote e *EJBs* ***Containers,*** responsáveis pela criação e gerenciamento de classes EJB.

Normalmente, os EJBs acessam bancos de dados e são acessados por aplicações WEB Java, que usam direta ou indiretamente as tecnologias *JSP* e *Servlet*. Essas últimas, por sua vez, são acessíveis por clientes HTTP, como navegadores (*browsers*), por exemplo. Quando um EJB é solicitado junto ao *EJB Container,* este decide se cria um novo ou se reaproveita um existente antes de enviar a quem solicitou. Esse processo encontra-se ilustrado na Figura Funcionamento dos EJBs no AS.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

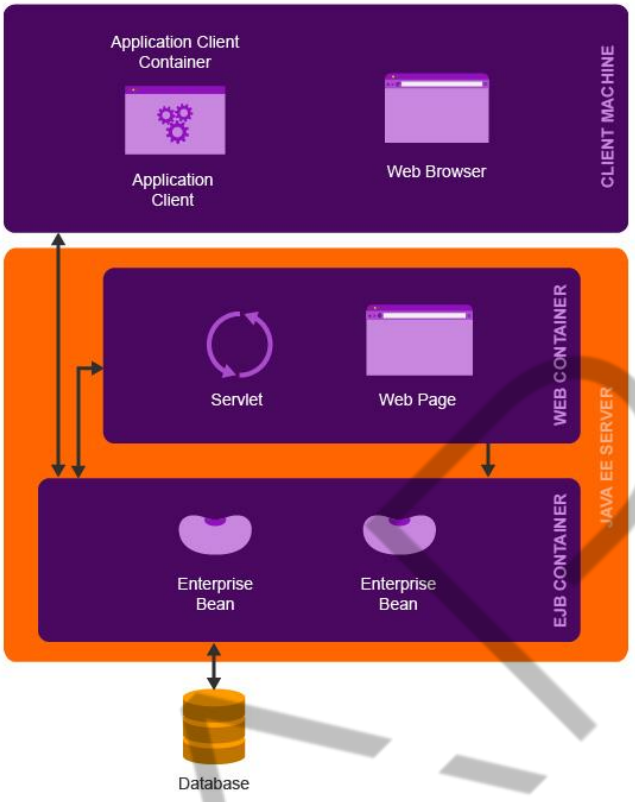


Figura 6.1 – Funcionamento dos EJBs no AS
Fonte: Oracle (2017)

Existem atualmente dois tipos de EJB: ***Session Beans*** e ***Message Driven Beans***. Em versões anteriores dos EJBs, existiam ainda os **Entity Beans**, que eram as classes de ORM que vimos em capítulos anteriores. Os Message Driven Beans não serão estudados neste capítulo, pois são usados para soluções bem específicas, fora do contexto desta disciplina. Os Session Beans, assunto principal deste capítulo, possuem uma aplicação bem geral, podendo ser utilizados para resolver quase todo tipo de problema. A seguir, estudaremos os tipos de EJB Session Beans.

6.1.1 Session Beans

Os **Session Beans** são classes Java aparentemente comuns, porém são criados, reaproveitados e descartados pelo AS onde o projeto EJB está instalado. A vantagem disso é que funcionalidades complexas, como controle de transações e acesso a conexões de banco de dados, ficam bem abstraídas, restando ao

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

desenvolvedor apenas indicar se quer usar este ou aquele recurso por meio de anotações (*annotations*).

Existem três tipos de Session Beans: ***Stateful***, ***Stateless*** e ***Singleton***. Veja a seguir o detalhamento de cada um deles.

Stateful Session Beans**:** é um Bean **com estado de sessão.** Sua característica é **manter o estado conversacional para um cliente específico**. Ou seja, enquanto a sessão de quem o solicitou estiver ativa, os valores de seus atributos se mantêm. Em outras palavras, **uma mesma instância** de um Stateful Bean acompanha a sessão de quem o solicitou.

Isso significa que devemos ter muito cuidado com esse tipo de Bean, pois, por exemplo, se 10 mil clientes acessarem uma certa funcionalidade que exige o uso de um Stateful Bean, 10 mil instâncias dele serão criadas pelo *Container*. E mantidas por ele até que as sessões sejam encerradas.

Para definir um Bean com estado de sessão, basta usar a anotação **@javax.ejb.Stateful** na definição da classe, como pode ser visto no Código-fonte Exemplo de Session Bean Stateful.

```
// pacote
import javax.ejb.Stateful;

@Stateful
public class TipoEstabelecimentoService {

    public TipoEstabelecimento recuperar(Integer id) {
        // código que retorna um TipoEstabelecimento
    }
}
```

Código-fonte 6.1 – Exemplo de Session Bean Stateful
Fonte: Elaborado pelo autor (2017)

O comportamento do estado de sessão de um Stateful Bean encontra-se ilustrado na figura a seguir. Nela, observe que a **mesma instância** do Bean acompanha o cliente por toda a sessão.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

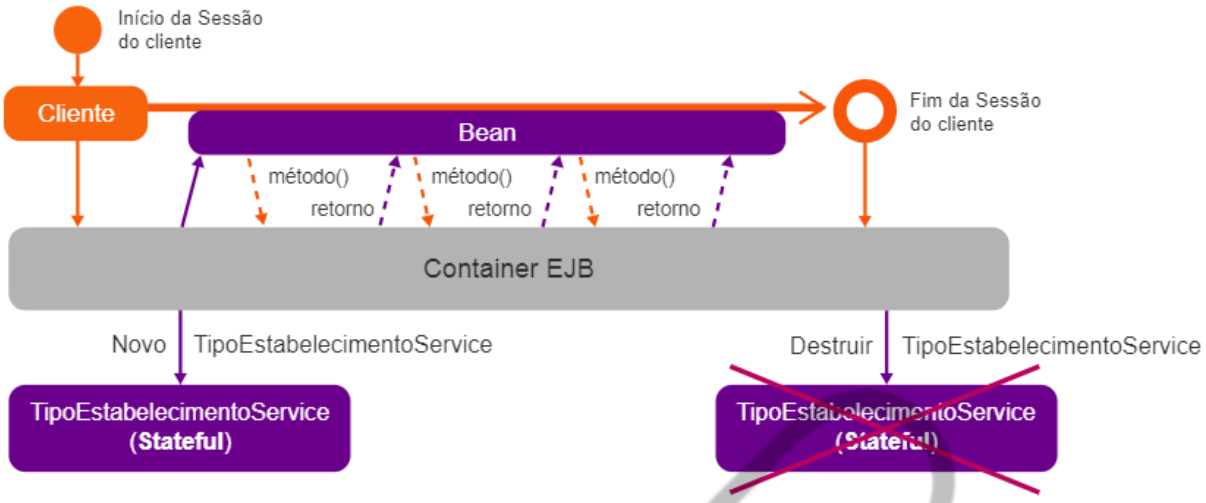


Figura 6.2 – Funcionamento de um Stateful Bean numa sessão do cliente
Fonte: Elaborado pelo autor (2017)

***Stateless Session Beans*:** É um Bean **sem estado de sessão.** Sua característica é **NÃO manter o estado conversacional para um cliente específico**. Ou seja, numa mesma sessão, quem o solicitou pode lidar com **diferentes instâncias** do mesmo Bean durante a sessão.

Isso significa que se, por exemplo, 10 mil clientes acessarem uma certa funcionalidade que exige o uso de um Stateless Bean, um número de 1 a *N* instâncias desse Bean será criado, conforme critério do *Container*, e **compartilhado** entre esses clientes.

Para definir um Bean sem estado de sessão, basta usar a anotação **@javax.ejb.Stateful** na definição da classe, como pode ser visto no exemplo do Código-fonte Exemplo de Session Bean Stateless.

```
// pacote
import javax.ejb.Stateless;

@Stateless
public class TipoEstabelecimentoService {

    public TipoEstabelecimento recuperar(Integer id) {
       // código que retorna um TipoEstabelecimento
    }
}
```

Código-fonte 6.2 – Exemplo de Session Bean Stateless
Fonte: Elaborado pelo autor (2017)

O comportamento do estado de sessão de um Stateful Bean encontra-se ilustrado na figura a seguir. Nela, observe que as **várias instâncias** do Bean podem

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

acompanhar o cliente durante sua sessão e que o Bean não necessariamente é destruído depois do encerramento da sessão do cliente.

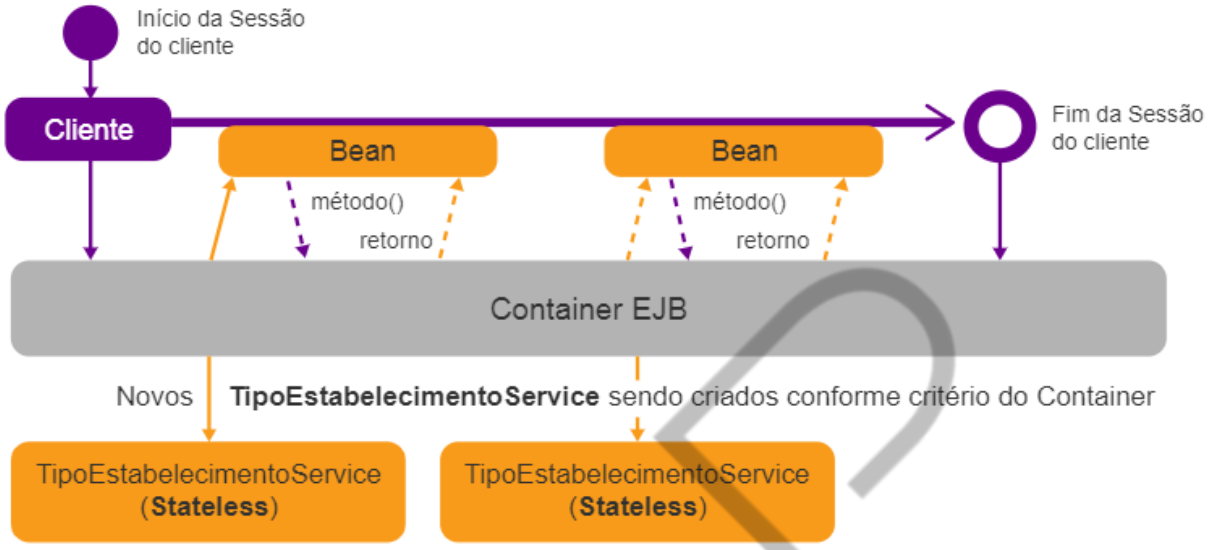


Figura 6.3 – Funcionamento de um Stateless Bean numa sessão de cliente
Fonte: Elaborado pelo autor (2017)

Singleton Session Beans*:** é um Bean que possui uma **única instância compartilhada por todos os clientes que o solicitam.** Sua característica é **manter o estado conversacional para todos os clientes do *Container. Ou seja, todos os clientes que solicitam um Bean desse tipo recebem a **mesma instância**.

Isso significa que, com esse tipo de Bean, se, por exemplo, 10 mil clientes acessarem uma certa funcionalidade que exige o uso de um Singleton Bean, **a mesma instância desse Bean será compartilhada** pelos 10 mil clientes.

Para definir um Bean Singleton, basta usar a anotação **@javax.ejb.Singleton** na definição da classe, como pode ser visto no exemplo do Código-fonte "Exemplo de Session Bean Singleton".

```
// pacote
import javax.ejb.Singleton;

@Singleton
public class TipoEstabelecimentoService {

    public TipoEstabelecimento recuperar(Integer id) {
       // código que retorna um TipoEstabelecimento
    }
}
```

Código-fonte 6.3 – Exemplo de Session Bean Singleton
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O comportamento do estado de sessão de um Singleton Bean está ilustrado na figura a seguir. Nela, observe que **a mesma instância** do Bean é usada por todos os clientes que solicitam o mesmo tipo de Bean Singleton.

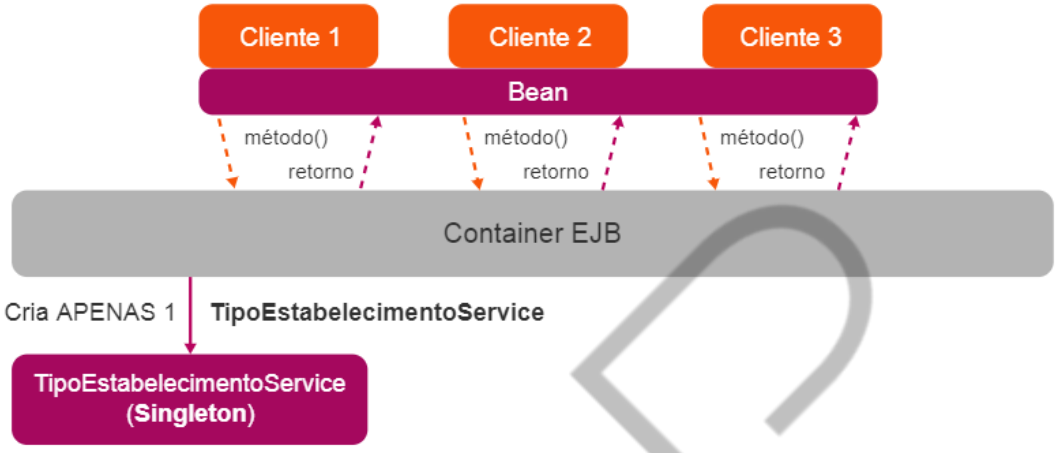


Figura 6.4 – Funcionamento de um Singleton Bean
Fonte: Elaborado pelo autor (2017)

Resumindo: para uma classe Java ser um EJB Session Bean, ela precisa estar anotada com **@javax.ejb.Staleful**, **@javax.ejb.Stateless** ou **@javax.ejb.Singleton**.

6.1.2 Chamadas assíncronas

Os **Session Beans** permitem a execução de **métodos de forma assíncrona**. Isso significa que podemos solicitar que um método inicie sua execução, mas não precisamos aguardar sua conclusão. Essa funcionalidade é muito útil em situações nas quais não precisamos aguardar o encerramento de um método para prosseguir um determinado fluxo. Exemplos de chamadas assíncronas do cotidiano:

- **Compras on-line pagas com cartão de crédito**. Note que, quando realizamos uma compra no cartão, ocorre imediatamente a criação do pedido e você já vê uma tela de conclusão de compra. Porém, alguns instantes depois, você recebe um e-mail dizendo algo como *"aguardando aprovação de pagamento"*. Mais um tempo e você recebe outro e-mail confirmando (ou não) o pagamento no cartão. Certamente, a conclusão do pedido enviou uma solicitação **assíncrona** para algum componente que se comunica com a operadora de cartão. Não fosse isso, o encerramento de sua compra poderia levar até minutos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Envio de TED bancário**. Você já deve ter enviado ou recebido TEDs, certo? Se sim, notou que o valor sai no mesmo instante da conta de origem, mas não cai imediatamente na conta de destino. Pode levar até 1 hora para chegar ao destino ou para a operação ser cancelada (por divergência de dados, por exemplo). Porém, você não fica no *home banking* ou aplicativo esperando todo esse tempo com uma ampulheta e uma mensagem "aguarde, enviando TED". Isso porque a solicitação de envio TED é feita de forma **assíncrona**.

Ou seja, chamadas assíncronas devem ser feitas quando algum procedimento pode demorar muito e não é realmente necessário aguardar sua conclusão imediatamente, como é o caso dos exemplos recém-citados.

Há duas formas de configurar chamadas assíncronas em Session Beans: Fazendo com que todos os métodos do Bean sejam executados de forma assíncrona ou configurando apenas um ou mais métodos para isso. Em ambos os casos, usamos a anotação **@javax.ejb.Asynchronous**.

Para que todos os métodos de uma classe sejam executados de forma assíncrona, a anotação **@javax.ejb.Asynchronous** deve ser colocada **na definição da classe**, como no exemplo do Código-fonte "Exemplo de Bean com todas as chamadas assíncronas".

```
// pacote
import javax.ejb.Stateless;
import javax.ejb.Asynchronous;

@Stateless
@Asynchronous
public class TipoEstabelecimentoService {

    public TipoEstabelecimento recuperar(Integer id) {
       // código que retorna um TipoEstabelecimento
    }

    public void excluir(Integer id) {
       // código que exclui um TipoEstabelecimento
    }
}
```

Código-fonte 6.4 – Exemplo de Bean com todas as chamadas assíncronas
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No exemplo anterior, tanto o método **recuperar()** quanto o método **excluir()** serão executados de forma assíncrona. Ou seja, quem os invocar não ficará aguardando sua conclusão para prosseguir para a próxima linha de código.

Para que apenas métodos específicos de uma classe sejam executados de forma assíncrona, a anotação **@javax.ejb.Asynchronous** deve ser colocada **na definição do método**, como no exemplo do Código-fonte "Exemplo de Bean com método assíncrono".

```
// pacote
import javax.ejb.Stateless;
import javax.ejb.Asynchronous;

@Stateless

public class TipoEstabelecimentoService {

    public TipoEstabelecimento recuperar(Integer id) {
       // código que retorna um TipoEstabelecimento
    }

    @Asynchronous
    public void excluir(Integer id) {
       // código que exclui um TipoEstabelecimento
    }
}
```

Código-fonte 6.5 – Exemplo de Bean com método assíncrono
Fonte: Elaborado pelo autor (2017)

No exemplo anterior, apenas o método **excluir()** será executado de forma assíncrona. Ou seja, quem o invocar não ficará aguardando sua conclusão para prosseguir para a próxima linha de código.

6.1.3 Métodos agendados (*Schedule*)

Imagine que você precise que algum procedimento ocorra uma vez por dia, sempre à meia-noite. Ou ainda que alguma atualização junto de um sistema externo deva ocorrer a cada 30 minutos. Para situações como essas, pode ser usada a funcionalidade de ***Schedule*** (agendamento) do EJB.Para tornar um método agendado, basta usar a anotação **@javax.ejb.Schedule** na sua definição. Vide o exemplo do Código-fonte "Exemplo de Bean com método agendado".

```
// pacote
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
import javax.ejb.Singleton;
import javax.ejb.Schedule;

@Singleton
public class TipoEstabelecimentoService {

    @Schedule(dayOfWeek="*", hour="0")
    public void atualizarTabelaTodaMeiaNoite() {
       // código que atualiza a tabela
    }

}
```

Código-fonte 6.6 – Exemplo de Bean com método agendado
Fonte: Elaborado pelo autor (2017)

No exemplo, o método **atualizarTabelaTodaMeiaNoite()** será executado à 0 hora (meia-noite) de todos os dias da semana. Note que o Bean é um *Singleton*, mas poderia ser *Stateless* ou *Stateful*, embora seja muito incomum criar métodos agendados para esses dois últimos tipos.

Um detalhe importante: o método agendado só teria sua primeira execução agendada quando o Bean **TipoEstabelecimentoService** for criado pelo *Container*. E isso só ocorrerá quando algum cliente o solicitar. Hipoteticamente, caso a aplicação fique duas semanas no ar e ninguém solicitar um **TipoEstabelecimentoService**, o método **atualizarTabelaTodaMeiaNoite()** passará esse tempo todo sem ser executado. Caso você precise que um método fique agendado logo que a aplicação EJB iniciar, basta anotar a classe com **@javax.ejb.Startup**. Assim o Bean ficaria:

```
// pacote
import javax.ejb.Singleton;
import javax.ejb.Schedule;
import javax.ejb.Startup;

@Singleton
@Startup
public class TipoEstabelecimentoService {

    @Schedule(dayOfWeek="*", hour="0")
    public void atualizarTabelaTodaMeiaNoite() {
       // código que atualiza a tabela
    }

}
```

Código-fonte 6.7 – Exemplo de Bean com método agendado desde o início da execução do projeto
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Ok, mas quais são as opções de agendamento? Somente diário e numa determinada hora? Não. A anotação **@javax.ejb.Schedule** possui muitas opções de agendamento por meio dos atributos **second**, **minute**, **hour**, **dayOfWeek**, **dayOfMonth**, **month**, **year**, conforme a tabela a seguir.

Tabela 6.1 – Configurações de agendamento para a anotação "@javax.ejb.Schedule"

Atributo	Descrição	Valor-padrão	Valores e exemplos
second	Segundo(s) no(s) minuto(s)	0	Valores: **De 0 a 59** *** - todos os segundos do minuto** Exemplos de intervalos: **0-14** - do 1º ao 15º segundo **0,5,30** - 1º, 6º e 30º segundo ***/5** - a cada 5 segundos
minute	Minuto(s) na(s) hora(s)	0	Valores: **De 0 a 59** *** - todos os minutos da hora** Exemplos de intervalos: **0-14** - do 1º ao 15º minuto **0,5,30** - 1º, 6º e 30º minuto ***/5** - a cada 5 minutos
hour	Hora(s) no(s) dia(s)	0	Valores: **De 0 a 23** *** - todas as horas do dia** Exemplos de intervalos: **0-14** - das 0h às 14h **0,9,19** - às 0h, 9h e 19h ***/5** - a cada 5 segundos
dayOfWeek	Dia(s) na semana	*	Valores: **De 0 a 7** Onde: 2ª(1), 3ª(2), 4ª(3), 5ª(4), 6ª(5), Sáb.(6) e Dom.(7 ou 0).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

			Valores alternativos: **Sun, Mon, Tue, Wed, Thu, Fri, Sat** ***** - **todos os dias da semana** Exemplos de intervalos: **1-5** - de 2ª a 6ª **1,3,5** - 2ª, 4ª e 6ª
dayOfMonth	Dia(s) no(s) mês(es)	*	Valores: **De 1 a 31** Valores negativos: **Dias antes do mês**. Ex.: **-5** (5 dias antes do mês) Último dia do mês: **Last** ***** - **todos os dias do mês** Exemplos de intervalos: **1-10** - do dia 1 ao dia 10 **5,10,25** - dias 5, 10 e 25 ***/5** - a cada 5 dias
month	Mês(es) no(s) ano(s)	*	Valores: **De 1 a 12** Valores alternativos: **Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec** ***** - **todos os meses do ano** Exemplos de intervalos: **1-3** - de janeiro a março **6,12** - junho e dezembro ***/2** - a cada 2 meses
year	Ano(s)	*	Valores: **Qualquer ano do calendário** ***** - **todos os anos** Exemplos de intervalos:

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

			2017-2018 - de 2017 a 2018 **2018, 2022** - 2018 e 2022 ***/4** - a cada 4 anos

Fonte: Elaborado pelo autor (2017)

E se eu precisar que um método tenha dois tipos de agendamentos distintos. Por exemplo: durante a semana, preciso que execute às 12h e à 0h, mas, aos finais de semana, que seja executado só às 15h? Nesse caso, o método precisa estar anotado com **@javax.ejb.Schedules**, que exige um vetor de **@javax.ejb.Schedule**. Veja no código-fonte como isso poderia ser feito:

```
// pacote
import javax.ejb.Singleton;
import javax.ejb.Schedule;
import javax.ejb.Startup;

@Singleton
@Startup
public class TipoEstabelecimentoService {

    @Schedules({
      @Schedule(dayOfWeek="1-5", hour="0,12"),
      @Schedule(dayOfWeek="6-7", hour="15")
    })
    public void atualizarDeVezEmQuando() {
       // código que atualiza a tabela
    }

}
```

Código-fonte 6.8 – Exemplo de Bean
Fonte: Elaborado pelo autor (2017)

No exemplo, o método **atualizarDeVezEmQuando()** seria executado sempre à 0h e às 12h, de 2ª a 6ª, e às 15h, aos sábados e domingos.

6.2 Projeto EJB + JPA +JSP + Servlets

Vamos agora criar um projeto EJB que também usa o JPA, que aprendemos anteriormente. Vamos ver na prática como o *EJB Container* controla a criação de *Beans Stateless*, *Stateful* e *Singleton*, e como ele gerencia conexões com banco de dados. Tudo isso num projeto com Servlets e JSP simples para testar nossos EJBs.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O projeto conterá apenas duas funcionalidades: Consulta de “estabelecimentos” por nome e um “painel” com o total de consultas realizadas e de usuários ativos.

6.2.1 Instalando o *JBoss Tools* no *Eclipse*

Para que possamos prosseguir, é necessário instalar no ***Eclipse*** alguns componentes de um *plugin* chamado ***JBoss Tools***. Ele é de uso livre, criado e mantido pela ***Red Hat***.

Vá ao menu **Help > Eclipse Marketplace**. Na janela que abrir, pesquise por “jboss tools” e role até encontrar o item ***JBoss Tools <versão>***. Clique em ***Install***. Na próxima tela, evite instalar todos os componentes do *plugin*. A recomendação é você escolher apenas ***JBoss AS, Wildfly & EAP Server Tools*** e ***Maven Integration for Eclipse APT***.

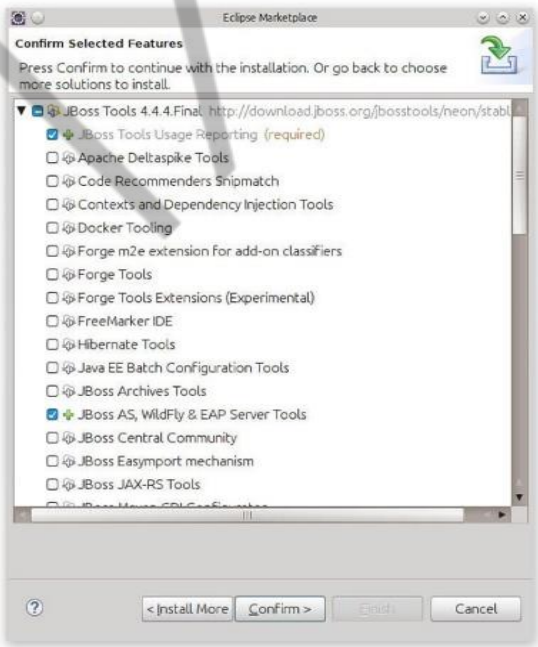


Figura 6.5 – Componente *JBoss AS, Wildfly & EAP Server Tools* do *JBoss Tools*.
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

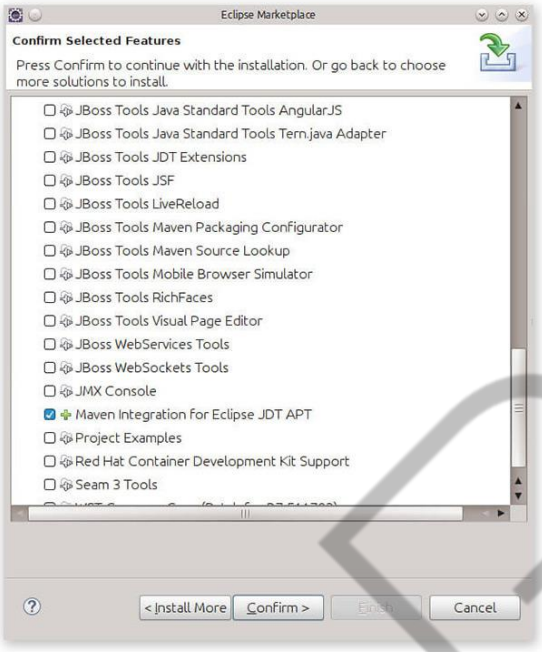


Figura 6.6 – Componente *Maven Integration for Eclipse APT* do *JBoss Tools*
Fonte: Elaborado pelo autor (2017)

As próximas telas serão de aceitação da licença de uso, confirmação para instalação de software de terceiro e permissão para reiniciar o *Eclipse*. Aceite ambas. Logo após reiniciar o *Eclipse*, o *JBoss Tools* perguntará se deseja que ele envie dados anônimos sobre o uso desse *plugin* para a *RedHat*. Responder ***Yes*** ou ***No*** não impede o uso da ferramenta, ficando a seu critério aceitar ou não isso.

6.2.2 Instalando o AS *WildFly*

Para que possamos testar nossos EJBs, precisamos que nosso projeto seja instalado num AS. Para nosso projeto, vamos instalar o ***WildFly 10***. Existe a versão **11**, mas, até a conclusão deste material, o *JBoss Tools* ainda tinha suporte apenas experimental para essa versão, por isso, a recomendação para a versão **10**. Acesse <**http://wildfly.org/downloads/**> e faça o download da versão estável mais recente da família 10.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

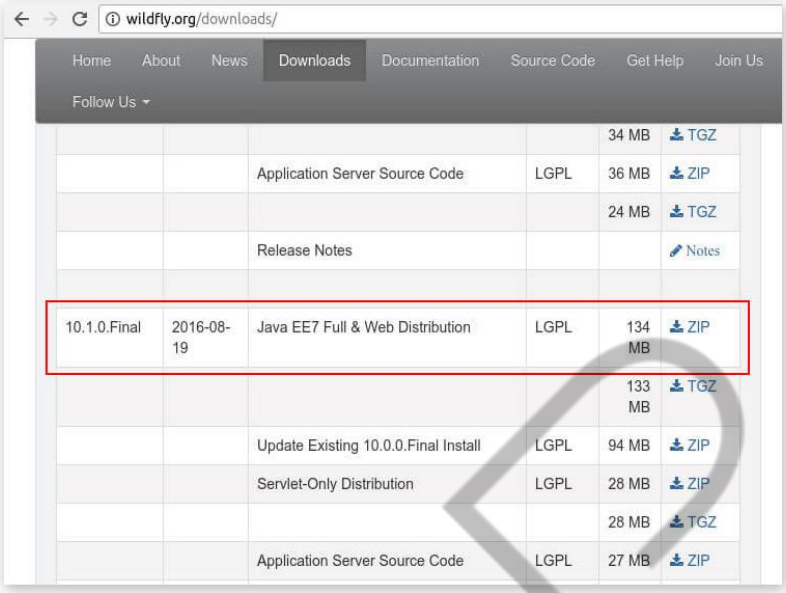


Figura 6.7 – Página de download do *WildFly*
Fonte: Elaborado pelo autor (2017)

Após fazer o download, descompacte o arquivo no diretório que preferir. Só não esqueça onde, pois precisará dele em vários momentos a seguir.

Procure pela aba ***Servers***. Se não estiver achando, mude a perspectiva para Java EE. Nela, se nenhum servidor estiver instalado no *Eclipse*, haverá um link azul para a instalação de um. De qualquer forma, basta clicar com o botão direito do mouse e escolher a opção **New > Server**.

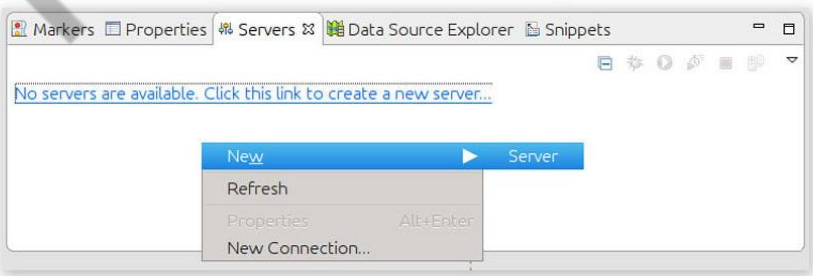


Figura 6.8 – Solicitando a instalação de um novo servidor no *Eclipse*
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Na lista de servidores, vá em **JBoss Community > WildFly 10**. Aceite as opções-padrão dos valores de ***Server's host name*** e ***Server name***. Na próxima tela, aceite também os valores-padrão.

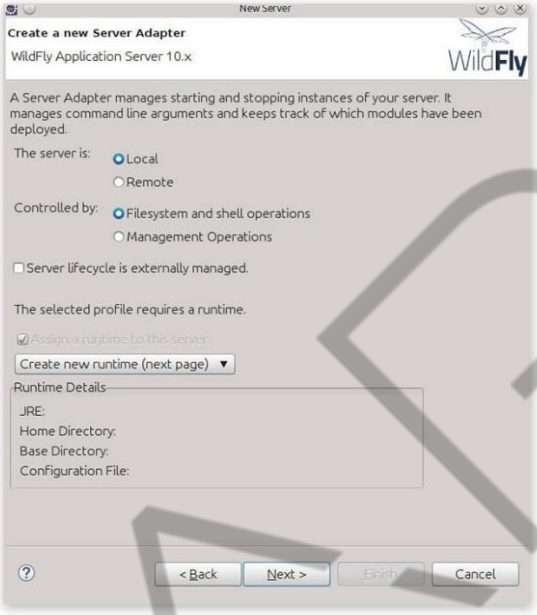


Figura 6.9 – Instalação de um *WildFly* no *Eclipse*
Fonte: Elaborado pelo autor (2017)

Na tela da sequência, você deve informar o caminho do diretório que foi criado quando você descompactou o ***WildFly 10***. Você pode manter os demais valores-padrão do assistente.

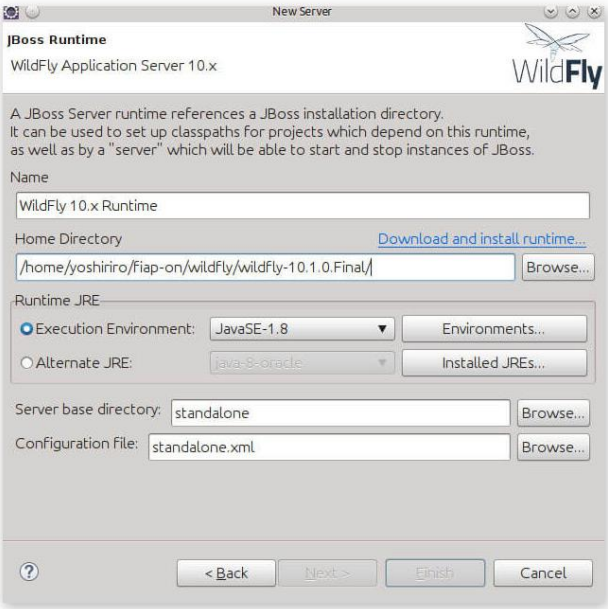


Figura 6.10 – Tela de indicação do caminho do *WildFly* no *Eclipse*
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Na última tela do assistente, será perguntado qual projeto Web ou EJB pretende instalar no ***WildFly***. Como ainda não criamos nosso projeto, pode clicar direto em ***Finish***. O ***WildFly 10.x*** passará a constar na aba ***Servers***. Clique nele e solicite que seja executado.

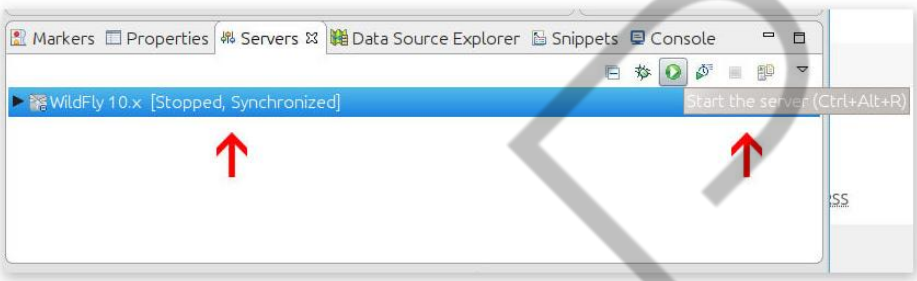


Figura 6.11 – *WildFly 10.x* na aba *Servers*
Fonte: Elaborado pelo autor (2017)

A inicialização do ***WildFly*** leva cerca de 20 segundos. Ao terminar sua inicialização, a aba ***Console*** do *Eclipse* conterá uma saída como a da figura Saída gerada no *Console* após a inicialização do *WildFly*.

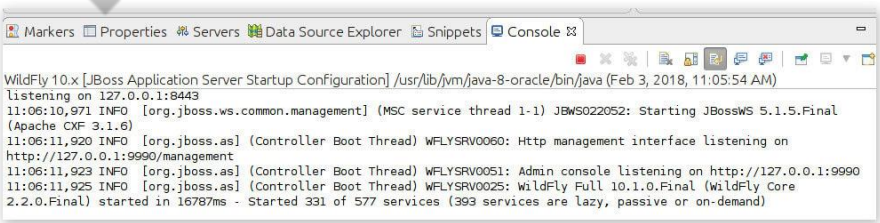


Figura 6.12 – Saída gerada no *Console* após a inicialização do *WildFly*
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Para ter total certeza de que o *WildFly* iniciou, acesse o endereço <**http://localhost:8080**> em algum navegador. Você deve ver uma página como a da figura Página inicial padrão do *WildFly*

Figura 6.13 – Página inicial padrão do *WildFly*
Fonte: Elaborado pelo autor (2017)

6.2.3 Criando um projeto EJB com Maven

Com o *JBoss Tools* e o *WildFly* instalados no *Eclipse*, já podemos criar nosso projeto EJB. Primeiro, solicite a criação de um novo projeto do tipo Maven. Na tela de seleção de *Archetypes*, escolha a opção **maven-archetype-webapp**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

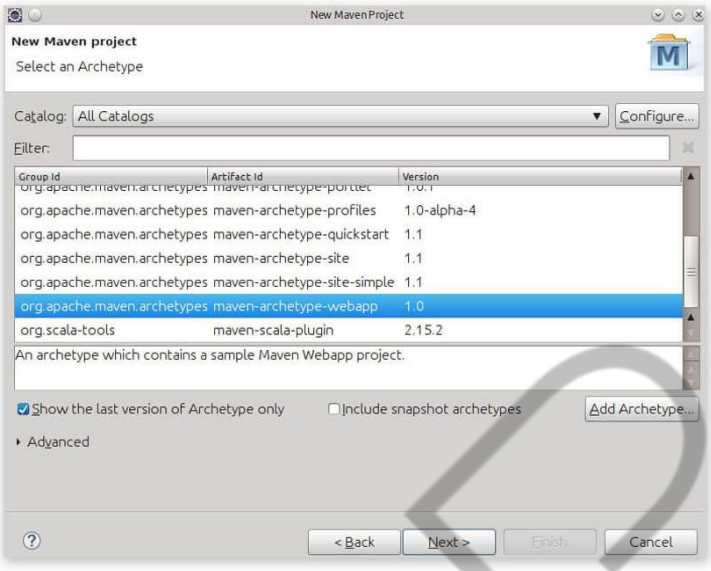


Figura 6.14 – Indicação do *archetype* do tipo *maven-archetype-webapp* no *Eclipse*
Fonte: Elaborado pelo autor (2017)

Na tela de indicação de dados do projeto *Maven*, indique os valores que preferir, mas recomendamos:

Group Id: br.com.fiap.smartcities

Artifact Id: smartcities-ejb

Package: br.com.fiap.smartcities.ejb

Uma vez criado, já pode ser adicionado no ***WildFly 10.x*** (na aba ***Servers***). Basta clicar com o botão direito do mouse sobre o servidor e escolher a opção ***Add and Remove***. Depois, apenas faça o projeto smartcities-ejb ir da coluna ***Avaliable*** para a ***Configured***. Se tudo correr bem, o projeto passará a constar dentro do servidor.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

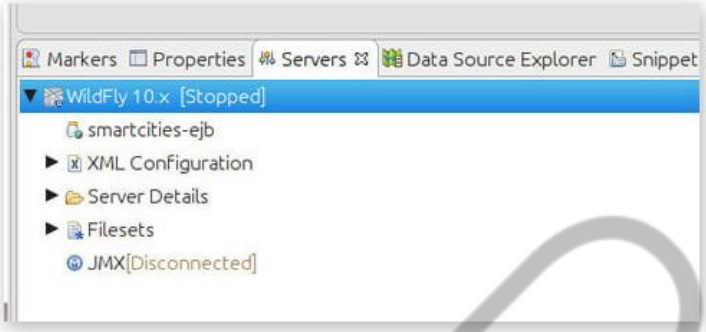


Figura 6.15 – Projeto *smartcities-ejb* no *WildFly 10.x*
Fonte: Elaborado pelo autor (2017)

Pare e inicie novamente o ***WildFly 10.x***. Então, acesse o endereço <**http://localhost:8080/smartcities-ejb**> em um navegador. Você deve ver uma página como a da figura Página do *smartcities-ejb* no navegador.

Figura 6.16 – Página do *smartcities-ejb* no navegador
Fonte: Elaborado pelo autor (2017)

Agora precisamos fazer um pequeno ajuste no projeto para continuarmos. O *archetype* que usamos para elaborar o projeto *Maven* não cria o diretório-padrão de

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

código-fonte de arquivos Java. Precisamos fazer isso manualmente. Basta criar um novo diretório (*folder*) em **src/main** chamado **java** (todas as letras minúsculas). Ao fazer isso, o item **src/main/java** passará a figurar imediatamente no item ***Java Resources*** do ***Project Explorer***. É nesse item que devem ser criados os pacotes e classes de nosso projeto, como em qualquer projeto Java feito com *Maven*.

6.2.3.1 Alterando o pom.xml

Precisamos alterar o **pom.xml** do projeto para incluir as dependências e configurações necessárias.

Inclua a tag ***<packaging>war</packaging>*** logo após a tag de versão do projeto.

Build

É necessário indicar toda uma configuração que está completa no Código-fonte "Configuração de *build* do projeto *smartcities-ejb*".

```
<build>
   <finalName>smartcities</finalName>
   <plugins>
      <plugin>
       <groupId>org.apache.maven.plugins</groupId>
       <artifactId>maven-compiler-plugin</artifactId>
       <version>3.3</version>
       <configuration>
         <source>1.8</source>
         <target>1.8</target>
       </configuration>
      </plugin>
      <plugin>
       <artifactId>maven-war-plugin</artifactId>
       <version>2.4</version>
       <configuration>
         <failOnMissingWebXml>false</failOnMissingWebXml>
       </configuration>
      </plugin>
   </plugins>
</build>
```

Código-fonte 6.9 – Configuração de *build* do projeto *smartcities-ejb*
Fonte: Elaborado pelo autor (2017)

Dependências

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vamos configurar apenas três dependências no projeto, descritas no Código-fonte “Dependências projeto *smartcities-ejb”*.

```
<dependencies>
  <dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>RELEASE</version>
    <scope>provided</scope>
  </dependency>
  <dependency>
    <groupId>javax</groupId>
    <artifactId>javaee-api</artifactId>
    <version>7.0</version>
    <scope>provided</scope>
  </dependency>
  <dependency>
    <groupId>org.jboss.spec.javax.servlet</groupId>
    <artifactId>jboss-servlet-api_3.1_spec</artifactId>
    <version>1.0.0.Final</version>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

Código-fonte 6.10 – Dependências projeto *smartcities-ejb*
Fonte: Elaborado pelo autor (2017)

Note que todas as dependências estão no escopo ***provided***. Isso significa que o *Maven* não vai compactá-las no **war** do projeto. É necessário fazer assim porque essas dependências correspondem a bibliotecas que já estão instaladas no ***WildFly*** por padrão. Por outro lado, se simplesmente não colocarmos essas dependências no **pom.xml**, o *Eclipse* não conseguirá importar classes de suas bibliotecas em nossas classes enquanto programamos.

6.2.3.2 Alterando o web.xml

Precisamos também alterar o arquivo **src/main/webapp/web.xml** do projeto para que ele permita usar a especificação 3.1 dos *Servlets*. Basta trocar seu conteúdo pelo do código-fonte:

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"

xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
         http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
          version="3.1">

</web-app>
```

Código-fonte 6.11 – Arquivo *web.xml* configurado para *Servlets* 3.1
Fonte: Elaborado pelo autor (2017)

6.2.3.3 Reaproveitando (copiando) classes do projeto JPA

Vamos copiar para este projeto as classes **Estabelecimento** e **TipoEstabelecimento**, ambas para o pacote **br.com.fiap.smartcities.domain**. São as mesmas do projeto que criamos nos capítulos anteriores.

6.2.3.4 Criando um *Data Source* no *WildFly*

Para que nosso projeto use a funcionalidade de gerenciamento de conexões e transações de um *EJB Container*, vamos configurar um ***Data Source*** no *WildFly*. Um ***Data Source*** é um componente que cria, administra e fecha conexões com um banco de dados. É ele que o *EJB Container* usa para criar ou reaproveitar conexões, abrir, confirmar ou dar *rollback* em transações e fechar conexões. Tudo isso sem que o programador se preocupe com essas operações no código de seu projeto EJB.

O primeiro passo é ir ao diretório de instalação do *WildFly* e criar um novo **módulo de biblioteca** (que é uma configuração própria de dependência do *WildFly*, como que um *Maven* dele) para o driver JDBC que o ***Data Source*** vai precisar.

Para isso, devemos entrar no diretório **modules/system/layers/base** do *WildFly* e criar um diretório chamado **jdbc** e, dentro deste, algum com o nome do servidor do banco de dados que pretende acessar (por exemplo: **Oracle** ou **MySQL**). E, dentro deste, criamos outro diretório com o nome **main**. Para não confundir...

Caso você queira acessar o banco **Oracle**, você agora deve possuir um diretório **modules/system/layers/base/jdbc/oracle/main/** no diretório do *WildFly*.

Caso você queira acessar o banco **MySQL**, você agora deve possuir um diretório **modules/system/layers/base/jdbc/mysql/main/** no diretório do *WildFly*.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Dentro do diretório **main**, cole o .jar do driver de conexão do banco que estiver usando. Ainda nesse diretório, crie um arquivo chamado **module.xml**. Esse arquivo contém uma configuração própria do *WildFly*. No Código-fonte "Exemplo de arquivo *module.xml* para Oracle", há um exemplo de seu conteúdo, caso você pretenda acessar o **Oracle**, e no Código-fonte "Exemplo de arquivo module.xml para *MySQL"*, um exemplo para **MySQL**.

```
<module xmlns="urn:jboss:module:1.3" name="jdbc.oracle">
    <resources>
      <resource-root path="ojdbc7.jar"/>
    </resources>
    <dependencies>
      <module name="javax.api"/>
      <module name="javax.transaction.api"/>
    </dependencies>
</module>
```

Código-fonte 6.12 – Exemplo de arquivo *module.xml* para Oracle
Fonte: Elaborado pelo autor (2017)

```
<module xmlns="urn:jboss:module:1.3" name="jdbc.mysql">
    <resources>
      <resource-root path="mysql-connector-java-5.1.44.jar"/>
    </resources>
    <dependencies>
      <module name="javax.api"/>
      <module name="javax.transaction.api"/>
    </dependencies>
</module>
```

Código-fonte 6.13 – Exemplo de arquivo module.xml para *MySQL*
Fonte: Elaborado pelo autor (2017)

Após configurar o módulo de biblioteca do driver JDBC necessário, podemos configurar o ***Data Source*** propriamente dito. Para isso, acesse o diretório **standalone/configuration/** do *WildFly* e edite o arquivo **standalone.xml**.

Procure pela tag **<datasources>**. Ao encontrá-la, provavelmente ela possuirá uma configuração de ***Data Source*** para um banco de dados H2. Não exclua essa configuração, pois o *WildFly* a usa internamente. Vamos incluir o ***Data Source*** de acordo com o driver JDBC que optamos usar. No Código-fonte "Exemplo de *Data Source* para Oracle no *standard.xml* do *WildFly"*, há um exemplo para o caso de **Oracle** e, no Código-fonte "Exemplo de *Data Source* para *MySQL* no *standard.xml* do *WildFly"*, um exemplo para o caso de **MySQL**.

```
<datasources>
  <datasource do h2...>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
  </datasource>
  <datasource      jndi-name="java:/smartcitiesDS"      pool-
name="smartcitiesDS" enabled="true" use-java-context="true">
    <connection-
url>jdbc:oracle:thin:@localhost:1521:fiap</connection-url>
    <driver>oracle</driver>
    <security>
      <user-name>root</user-name>
      <password>admin</password>
  </security>
  </datasource>
  <drivers>
    <driver do h2... >
    </driver>
    <driver name="oracle" module="jdbc.oracle">
      <driver-class>oracle.jdbc.driver.OracleDriver</driver-
class>
    </driver>
  </drivers>
</datasources>
```

Código-fonte 6.14 – Exemplo de *Data Source* para Oracle no *standard.xml* do *WildFly*
Fonte: Elaborado pelo autor (2017)

```
<datasources>
  <datasource do h2...>
  </datasource>
  <datasource      jndi-name="java:/smartcitiesDS"      pool-
name="smartcitiesDS" enabled="true" use-java-context="true">
    <connection-
url>jdbc:mysql://localhost:3306/fiap</connection-url>
    <driver>mysql</driver>
    <security>
      <user-name>root</user-name>
      <password>admin</password>
  </security>
  </datasource>
  <drivers>
    <driver do h2... >
    </driver>
    <driver name="mysql" module="jdbc.mysql">
      <driver-class>com.mysql.jdbc.Driver</driver-class>
    </driver>
  </drivers>
</datasources>
```

Código-fonte 6.15 – Exemplo de *Data Source* para *MySQL* no *standard.xml* do *WildFly*
Fonte: Elaborado pelo autor (2017)

Nos Códigos-fonte “Exemplo de *Data Source* para Oracle no *standard.xml* do *WildFly”* e “Exemplo de *Data Source* para *MySQL* no *standard.xml* do *WildFly”*, o nome do ***Data Source*** é o da tag **<datasource>**, que, em ambos, é

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

“java:/smartcitiesDS”. É por esse nome que ele será referenciado logo mais no projeto. As demais tags possuem valores familiares para você, já estudados quando falamos sobre **JDBC** e **JPA**.

6.2.3.5. Criando o persistence.xml

Com o ***Data Source*** configurado, podemos criar o **persistence.xml** do **JPA** do projeto EJB. Lembrando que ele deve ficar no diretório **src/mais/resources/META-INF** do projeto. Veja no Código-fonte “Arquivo *persistence.xml* do projeto *smartcities-ejb”* como ele ficaria:

```
<persistence ...>
<persistence-unit name="smartcities" transaction-type="JTA">

  <jta-data-source>java:/smartcitiesDS</jta-data-source>

<provider>org.hibernate.jpa.HibernatePersistenceProvider</pro
vider>


<class>br.com.fiap.smartcities.domain.Estabelecimento</class>

<class>br.com.fiap.smartcities.domain.TipoEstabelecimento</cl
ass>

  <exclude-unlisted-classes>true</exclude-unlisted-classes>

  <properties>
    <!-- propriedades do Hibernate -->
  </properties>

</persistence-unit>

</persistence>
```

Código-fonte 6.16 – Arquivo *persistence.xml* do projeto *smartcities-ejb*
Fonte: Elaborado pelo autor (2017)

Na tag **<persistence-unit>**, o valor de ***“transaction-type”*** agora é JTA. Isso indica que o controle das conexões e transações JPA do projeto será de responsabilidade do *EJB Container*.

Agora existe a tag **<jta-data-source>**, cujo conteúdo é exatamente o nome do ***Data Source*** que criamos seguindo os exemplos dos Códigos-fonte “Exemplo de

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Data Source para Oracle no *standard.xml* do *WildFly"* e "Exemplo de *Data Source* para *MySQL* no *standard.xml* do *WildFly"*.

Na tag **<properties>**, devem constar apenas propriedades específicas do hibernate, como ***"hibernate.hbm2ddl.auto"*** ou ***"hibernate.show_sql"***, por exemplo. Propriedades de conexão com banco serão ignoradas, uma vez que usamos a tag **<jta-data-source>**.

6.2.3.6 *Singleton Bean*: ContadorPesquisasService.

Nosso projeto terá um *Singleton Bean*, o **ContadorPesquisasService**. Ele terá um contador de usuários "ativos" e de pesquisas realizadas. Veja seu conteúdo no código-fonte:

```
package br.com.fiap.smartcities.ejb;

import javax.ejb.Singleton;

@Singleton
public class ContadorPesquisasService {

  private int pesquisas;
  private int usuarios;

  public void novoUsuario() {
       usuarios++;
  }

  public void usuarioSaiu() {
       usuarios--;
  }

  public void novaPesquisa() {
       pesquisas++;
  }

  // getters
}
```

Código-fonte 6.17 – Código do ContadorPesquisasService
Fonte: Elaborado pelo autor (2017)

Como essa classe está anotada com **@javax.ejb.Singleton**, todos que a solicitarem receberão a **mesma instância** sempre. Por isso, seus valores **usuarios** e **pesquisas** serão os mesmos para qualquer um que solicitá-los. Eles serão

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

atualizados também por qualquer um que invocar os métodos **novaPesquisa()**, **novoUsuario()** e **usuarioSaiu()**.

6.2.3.7 *Stateful Bean*: HistoricoPesquisasService.

Nosso projeto terá um *Stateful Bean*, o **HistoricoPesquisasService**. Ele terá um *Set* para registrar o histórico de pesquisas de uma sessão de usuário. Outro detalhe é que ele possui um atributo do tipo **ContadorPesquisasService**, usado para atualizar a quantidade de pesquisas realizadas. Veja seu conteúdo no Código-fonte "Código do HistoricoPesquisasService".

```
package br.com.fiap.smartcities.ejb;

import java.util.LinkedHashSet;
import java.util.Set;

import javax.ejb.EJB;
import javax.ejb.Stateful;

@Stateful
public class HistoricoPesquisasService {

  @EJB
  private ContadorPesquisasService contadorService;

  private Set<String> historico = new
LinkedHashSet<String>();

  public void registrarPesquisa(String termo) {
    this.historico.add(termo);
    this.contadorService.novaPesquisa();
  }

  // get do histórico
}
```

Código-fonte 6.18 – Código do HistoricoPesquisasService
Fonte: Elaborado pelo autor (2017)

Essa classe está anotada com **@javax.ejb.Stateful**, o que garante que um cliente que a invocou continuará com a mesma instância por toda a sua sessão. Se um mesmo cliente, mesmo a partir de classes diferentes, solicitar esse *Bean*, sempre receberá a mesma instância.

Outro detalhe é o uso da anotação **@javax.ejb.EJB** sobre o atributo do tipo **ContadorPesquisasService**. Isso faz com que o *EJB Container* injete uma instância

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

(no caso, sempre a mesma, pois ela é um *Singleton Bean*) de **ContadorPesquisasService** no atributo **contadorService**. Por isso, você não o vê instanciado manualmente no Código-fonte "Código do HistoricoPesquisasService", e mesmo assim a linha na qual está a chamada do método **novaPesquisa()** será executada sem erros.

O método **registrarPesquisa()** apenas solicita a inclusão do termo recém-pesquisado no **Set** de histórico de pesquisas da sessão. Usamos um **Set** e não um **List** para evitar automaticamente a repetição de termos já pesquisados no histórico.

6.2.3.8 *Stateless Bean*: EstabelecimentosService.

Nosso projeto terá um *Stateless Bean*, o **EstabelecimentosService**. Ele possui um método chamado **pesquisar()**, que recebe um termo e usa um atributo do tipo ***EntityManager*** para procurar esse terno no campo **"nome_estabelecimento"** da tabela **"estabelecimento"**. Veja seu conteúdo no Código-fonte "Código do EstabelecimentosService".

```
package br.com.fiap.smartcities.ejb;

import java.util.ArrayList;
import java.util.List;

import javax.ejb.Stateless;
import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;

import br.com.fiap.smartcities.domain.Estabelecimento;

@Stateless
public class EstabelecimentosService {

  @PersistenceContext
  private EntityManager entityManager;

  public List<String> pesquisar(String termo) {
    List<String> resultado = new ArrayList<String>();

    List<Estabelecimento>          estabelecimentos          =
this.entityManager
    .createQuery("select e from Estabelecimento e "
    + "where e.nome like :nome")
    .setParameter("nome", "%"+termo+"%").getResultList();
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    for (Estabelecimento estabelecimento : estabelecimentos)
{
      resultado.add(estabelecimento.getNome()   +   "   ("   +
estabelecimento.getTipo().getNome() + ")");
    }
    return resultado;
  }
}
```

Código-fonte 6.19 – Código do EstabelecimentosService
Fonte: Elaborado pelo autor (2017)

Essa classe está anotada com **@javax.ejb.Stateless**, o que garante que o *EJB Container* é que decidirá se criará uma nova instância ou enviará uma já existente a quem solicitar o uso desse *Bean*. Um mesmo cliente, numa mesma sessão, pode solicitar esse *Bean* várias vezes e receber diferentes instâncias dele.

A anotação **@javax.persistence.PersistenceContext** sobre o objeto do tipo **EntityManager** faz com que esse objeto tenha seu valor injetado pelo *EJB Container*. Isso significa que o **EntityManager** injetado é aberto, tem uma transação aberta e depois fechada, e, então, é fechado pelo *EJB Container*. Tudo isso de forma transparente, possibilitando ao desenvolvedor usar o **EntityManager** somente para a operação de consulta/inclusão/alteração/exclusão.

A consulta realizada pelo método **pesquisar()** devolve uma lista vazia com elementos contendo o nome do estabelecimento e seu tipo entre parênteses. Sua pesquisa é feita usando a cláusula ***like*** sobre o atributo **nome** da entidade **Estabelecimento**.

6.2.3.9 *Serlvet Listener*: SmartcitiesListener.

Nosso projeto terá um *Servlet Listener*, o **SmartcitiesListener**. Trata-se de um tipo especial de classe em projetos *Servlet* que “escuta” quando ocorrem diferentes eventos na aplicação. No caso, configuramos **o SmartcitiesListener** para “escutar” os eventos de **criação** e **encerramento** de sessão. Veja seu conteúdo no Código-fonte “Código do SmartcitiesListener”.

```
package br.com.fiap.smartcities.servlets;

import javax.naming.InitialContext;
import javax.naming.NamingException;
import javax.servlet.annotation.WebListener;
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
import javax.servlet.http.HttpSessionEvent;
import javax.servlet.http.HttpSessionListener;

import br.com.fiap.smartcities.ejb.ContadorPesquisasService;

@WebListener
public       class       SmartcitiesListener       implements
HttpSessionListener {

  public void sessionCreated(HttpSessionEvent se) {
    try {
      InitialContext ic =  new InitialContext();

      se.getSession().setAttribute(
       "estabelecimentosService",
ic.lookup("java:module/EstabelecimentosService"));

      se.getSession().setAttribute(
       "historicoService",
ic.lookup("java:module/HistoricoPesquisasService"));

      ContadorPesquisasService contadorService
       =                          (ContadorPesquisasService)
ic.lookup("java:module/ContadorPesquisasService");

      contadorService.novoUsuario();

      se.getSession().setAttribute(
       "contadorService", contadorService);

    } catch (NamingException e) {
        e.printStackTrace();
    }
  }

  public void sessionDestroyed(HttpSessionEvent se) {
    ContadorPesquisasService contadorService =
     (ContadorPesquisasService)
se.getSession().getAttribute("contadorService");

    contadorService.usuarioSaiu();
  }
}
```

Código-fonte 6.20 – Código do SmartcitiesListener
Fonte: Elaborado pelo autor (2017)

Essa classe está anotada com **@javax.servlet.annotation.WebListener**, e é isso que a faz ser um *Listener*. Seu método **sessionCreated()** será invocado automaticamente sempre que um novo cliente acessar a aplicação **smatcities-ejb**

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

via navegador. Seu método **sessionDestoryed()** será invocado automaticamente sempre que uma sessão da aplicação **smatcities-ejb** for encerrada.

No método **sessionCreated()**, é instanciado um objeto do tipo **javax.naming.InitialContext** que serve, entre outras coisas, para solicitar instâncias de EJB do projeto por meio de seu método **lookup()**. Os nomes pelos quais os EJBs são encontrados pelo método **lookup()** são **"java:module/<Nome da Classe>"** (isso quem definiu foi o *WildFly*). Após solicitarmos os três *Beans* existentes no projeto, incluímos eles no escopo da sessão recém-criada. Apenas a instância de **ContadorPesquisasService** tem um método invocado aqui, o que incrementa a quantidade de "usuário ativos".

Por que solicitamos os três *Beans* no mesmo momento, da mesma forma e sempre quando uma sessão é criada? Porque assim é possível testar e verificar de forma simples os ciclos de vida dos três tipos de *Session Beans* que estudamos aqui. Todos os três *Beans* (cada um de um tipo) são solicitados ao mesmo tempo, porém, o **ContadorPesquisasService** sempre chegará a uma mesma instância para todos; o **HistoricoPesquisasService** chegará a uma instância nova, visto que é o momento em que uma sessão acaba de ser criada; e o **EstabelecimentoService** pode chegar como uma nova instância ou uma já existente, a critério do *EJB Container*.

No método **sessionDestroyed()**, apenas invocamos o método que "avisa" que um usuário "saiu" do sistema por meio da invocação do método **usuarioSaiu()** da **ContadorPesquisasService**.

6.2.3.10 ContadorServlet e contador.Jsp

Nosso projeto terá um *Servlet* chamado **ContadorServlet**. Ele apenas recupera o **ContadorPesquisasService** do escopo de sessão e joga seus dados de números de usuários ativos e pesquisas realizadas para o escopo de requisição. Ao final, ele redireciona seu resultado para a página **contador.jsp**. Veja seu conteúdo no Código-fonte "Código do ContadorServlet".

```
package br.com.fiap.smartcities.servlets;

import br.com.fiap.smartcities.ejb.ContadorPesquisasService;
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
// demais imports

@WebServlet(value = "/contador")
public class ContadorServlet extends HttpServlet {

  @Override
  protected void doGet(HttpServletRequest req,
HttpServletResponse resp) throws ServletException,
IOException {

    ContadorPesquisasService contadorService =
      (ContadorPesquisasService)
req.getSession().getAttribute("contadorService");

    req.setAttribute("usuarios",
contadorService.getUsuarios());
    req.setAttribute("pesquisas",
contadorService.getPesquisas());

    RequestDispatcher dispacher =
      req.getRequestDispatcher("contador.jsp");

    dispacher.forward(req, resp);
  }
}
```

Código-fonte 6.21 – Código do ContadorServlet
Fonte: Elaborado pelo autor (2017)

Para testar esse *Servlet*, a URL no navegador deve ser <**http://localhost:8080/smartcities-ejb/contador**>.

A página de resultado, **contador.jsp**, deve estar em **src/main/webapp/**. Seu conteúdo está a partir da tag **<body>**.

```
     <h2>Contador de Pesquisas</h2>

     <p>Existem <b>${usuarios}</b> usuários realizando
pesquisas.</p>

     <p>Já foram feitas <b>${pesquisas}</b> pesquisas.</p>
```

Código-fonte 6.22 – Código do *contador.jsp*
Fonte: Elaborado pelo autor (2017)

Após invocar salvar tudo e reiniciar o *WildFly*, caso teste a URL desse *Servlet*, o resultado deve ser uma página como a da figura a seguir.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 6.17 – Página do *ContadorServlet* no navegador
Fonte: Elaborado pelo autor (2017)

6.2.3.11 EstabelecimentosServlet e estabelecimentos.jsp

Nosso projeto terá um *Servlet* chamado **EstabelecimentosServlet**. Ele recupera o **EstabelecimentosService** do escopo de sessão e o utiliza para realizar a pesquisa de estabelecimento a partir do parâmetro de requisição **termo**. Também recupera o **HistoricoPesquisasService** para invocar seu **registrarPesquisa()**.

Para facilitar o código do JSP de resposta, os resultados foram colocados em tags HTML **<li>** de acordo com a quantidade de registros encontrados e de itens no histórico.

Os resultados da pesquisa e os itens do histórico de pesquisa são colocados no escopo de requisição para que possam ser acessíveis no **estabelecimentos.jsp**, cujo conteúdo está no código-fonte:

```
Package br.com.fiap.smartcities.servlets;

import br.com.fiap.smartcities.ejb.EstabelecimentosService;
import br.com.fiap.smartcities.ejb.HistoricoPesquisasService;
// demais imports

@WebServlet(value = "/estabelecimentos")
public class EstabelecimentosServlet extends HttpServlet {

  @Override
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
  protected      void      doGet(HttpServletRequest      req,
HttpServletResponse     resp)     throws     ServletException,
IOException {

      EstabelecimentosService estabelecimentosService =
     (EstabelecimentosService) req.getSession()
     .getAttribute("estabelecimentosService");

      HistoricoPesquisasService historicoService =
     (HistoricoPesquisasService) req.getSession()
     .getAttribute("historicoService");

      String termo = req.getParameter("termo");

      this.pesquisar(termo, req, estabelecimentosService);
      this.registrarHistorico(termo, req, historicoService);

      RequestDispatcher dispacher =
     req.getRequestDispatcher("estabelecimentos.jsp");
      dispacher.forward(req, resp);

  }

  private  void  pesquisar(String  termo,  HttpServletRequest
req, EstabelecimentosService estabelecimentosService) {
    List<String>             resultadoPesquisa             =
estabelecimentosService.pesquisar(termo);

    StringBuilder sbResultado = new StringBuilder();
    for (String resultado : resultadoPesquisa) {
         sbResultado.append("<li>" + resultado + "</li>");
    }

    req.setAttribute("termo", termo);
    req.setAttribute("encontrados",
resultadoPesquisa.size());
    req.setAttribute("resultado", sbResultado.toString());
  }

  private      void      registrarHistorico(String      termo,
HttpServletRequest       req,       HistoricoPesquisasService
historicoService) {
    historicoService.registrarPesquisa(termo);

    Set<String> pesquisas = historicoService.getHistorico();

    StringBuilder sbHistorico = new StringBuilder();
    for (String pesquisa : pesquisas) {
      sbHistorico.append("<li><i>" + pesquisa + "</i></li>");
    }
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    req.setAttribute("historico", sbHistorico);
  }
}
```

Código-fonte 6.23 – Código do *EstabelecimentosServlet*
Fonte: Elaborado pelo autor (2017)

Para testar esse *Servlet*, a URL no navegador deve ser **http://localhost:8080/smartcities-ejb/estabelecimentos?termo=<termo>**.

A página de resultado, **contador.jsp**, deve estar em **src/main/webapp/**. Seu conteúdo está a partir da tag **<body>**, no Código-fonte "Código do *estabelecimentos.jsp"*.

```
    <h2>Contador de Pesquisas</h2>

    <p>Existem <b>${usuarios}</b> usuários realizando
pesquisas.</p>

    <p>Já foram feitas <b>${pesquisas}</b> pesquisas.</p>
```

Código-fonte 6.24 – Código do *estabelecimentos.jsp*
Fonte: Elaborado pelo autor (2017)

Após invocar salvar tudo e reiniciar o *WildFly*, caso teste a URL desse *Servlet*, o resultado deve ser uma página como a da Figura "Página do *EstabelecimentosServlet* no navegador".

Figura 6.18 – Página do *EstabelecimentosServlet* no navegador
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6.3 Interfaces locais e remotas de um *Session Bean*

No projeto que acabamos de descrever, todos os *EJBs Session Beans* eram acessados apenas localmente por *Servlets* do próprio projeto. Porém, é possível expor *Session Beans* para que possam ser acessados remotamente. A seguir, faremos pequenas alterações no projeto EJB, assim como criaremos um projeto Java que funcionará como cliente do outro projeto.

6.3.1 Expondo o EstabelecimentosService para acesso remoto

Para que um *Session Bean* possa ser acessível remotamente, sua classe deve estar anotada com **@javax.ejb.Remote,** cujo valor deve ser uma interface que contenha os métodos que queremos expor para clientes remotos. Assim, para usar essa anotação, precisamos criar essa interface. Vamos chamá-la de **EstabelecimentosServiceRemote**, cujo conteúdo pode ser visto no código-fonte:

```
package br.com.fiap.smartcities.ejb;

public interface EstabelecimentosServiceRemote {

     List<String> pesquisar(String termo);
}
```

Código-fonte 6.25 – Código da interface EstabelecimentosServiceRemote
Fonte: Elaborado pelo autor (2017)

Com a interface criada, podemos indicar que ela será a “versão remota” do **EstabelecimentosService**. Assim, sua definição ficaria como a do código-fonte:

```
// pacote e imports

import javax.ejb.Remote;

@Stateless
@Remote(EstabelecimentosServiceRemote.class)
public class EstabelecimentosService {
 // mesmo conteúdo de antes
}
```

Código-fonte 6.26 – Interface EstabelecimentosServiceRemote indicada como “versão remota”
Fonte: Elaborado pelo autor (2017)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6.3.2 Expondo o EstabelecimentosService para acesso local

Quando definimos uma “versão remota” de um *Session Bean*, temos que definir explicitamente sua “versão local”, caso contrário passa a ficar inacessível no mesmo projeto. Para isso, o *Session Bean* deve estar anotado com **@javax.ejb.Local**, cujo valor deve ser uma interface que contenha os métodos que queremos expor para clientes locais. Assim, para usar essa anotação, precisamos criar essa interface. Vamos chamá-la de **EstabelecimentosServiceLocal**, cujo conteúdo está no Código-fonte “Código da interface EstabelecimentosServiceLocal”.

```
package br.com.fiap.smartcities.ejb;

public interface EstabelecimentosServiceLocal {

     List<String> pesquisar(String termo);
}
```

Código-fonte 6.27 – Código da interface EstabelecimentosServiceLocal
Fonte: Elaborado pelo autor (2017)

Com a interface criada, podemos indicar que ela será a “versão local” do **EstabelecimentosService**. Assim, sua definição ficaria como a do código:

```
// pacote e imports
import javax.ejb.Remote;
import javax.ejb.Local;

@Stateless
@Remote(EstabelecimentosServiceRemote.class)
@Local(EstabelecimentosServiceLocal.class)
public class EstabelecimentosService {
 // mesmo conteúdo de antes
}
```

Código-fonte 6.28 – Interface EstabelecimentosServiceLocal indicada como “versão local”
Fonte: Elaborado pelo autor (2017)

Com essa alteração, precisamos atualizar o *Listener* **SmartcitiesListener**, alterando a linha **ic.lookup("java:module/EstabelecimentosService")** para **ic.lookup("java:module/EstabelecimentosService!br.com.fiap.smartcities.ejb.EstabelecimentosServiceLocal")**.

Uma última alteração é no **EstabelecimentosServlets**, em que devemos trocar todas as referências de **EstabelecimentosService** para **EstabelecimentosServiceLocal**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6.3.3 Cliente para o projeto EJB

Vamos criar um projeto Java *Maven* simples que poderá ser executado em outro computador na mesma rede do projeto EJB que acessará os métodos de **EstabelecimentosServiceRemote**.

Ao criar um projeto *Maven*, atualize suas dependências para que o projeto consiga funcionar como cliente EJB, conforme o código-fonte:

```
<dependency>
  <groupId>org.jboss</groupId>
  <artifactId>jboss-remote-naming</artifactId>
  <version>2.0.4.Final</version>
  <scope>compile</scope>
</dependency>
<dependency>
  <groupId>org.wildfly</groupId>
  <artifactId>wildfly-controller-client</artifactId>
  <version>8.2.1.Final</version>
</dependency>
```

Código-fonte 6.29 – Dependências necessárias para criar um cliente EJB
Fonte: Elaborado pelo autor (2017)

Essas dependências podem variar, conforme o servidor usado. As que usamos aqui são compatíveis com o *WildFly*.

Para acessar a “versão remota” do **EstabelecimentosService**, copiaremos a interface **br.com.fiap.smartcities.ejb.EstabelecimentosServiceRemote** do projeto EJB para este.

Agora vamos criar uma classe Java simples, com método **main()**, que acessa o EJB remoto configurado anteriormente. Vamos chamá-la de **ClienteEJB**, cujo conteúdo está no Código-fonte “Classe Java que funciona como Cliente EJB”.

```
package br.com.fiap.smartcities.clientejb;

import java.util.Properties;

import javax.naming.Binding;
import javax.naming.Context;
import javax.naming.InitialContext;
import javax.naming.NamingEnumeration;
import javax.naming.NamingException;

import
br.com.fiap.smartcities.ejb.EstabelecimentosServiceRemote;
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
public class ClienteEJB
{
  public static void main( String[] args ) throws Exception
  {
    Properties env = new Properties();
    env.put(Context.INITIAL_CONTEXT_FACTORY,
"org.jboss.naming.remote.client.InitialContextFactory");
    env.put(Context.PROVIDER_URL, "remote://127.0.0.1:8080");
    env.put(Context.SECURITY_PRINCIPAL, "fiap");
    env.put(Context.SECURITY_CREDENTIALS, "senh@f1ap");
    env.put("jboss.naming.client.ejb.context", true);

    InitialContext ctx = new InitialContext(env);

    EstabelecimentosServiceRemote service =
    (EstabelecimentosServiceRemote)
ctx.lookup("ejb:/smartcities-
ejb/EstabelecimentosService!br.com.fiap.smartcities.ejb.Estab
elecimentosServiceRemote");

    List<String> resultado = service.pesquisar("FIAP");
    System.out.println(resultado.size()     +     "     itens
encontrados: ");
    for (String item : resultado) {
      System.out.println(item);
    }
  }
}
```

Código-fonte 6.30 – Classe Java que funciona como Cliente EJB
Fonte: Elaborado pelo autor (2017)

Notou que há configurações de “login” e “senha” na classe? Ocorre que, por padrão, os EJBs do *WildFly* são “protegidos” pelo seu *EJB Container*. Para definir um login e senha, você deve parar o *WildFly*, entrar no seu diretório de instalação, depois no diretório bin e executar o comando **add-user.bat -a -u 'fiap' -p 'senh@f1ap'** ou **add-user.sh -a -u 'fiap' -p 'senh@f1ap'**, caso esteja usando Linux ou MacOS. Os valores ***‘fiap’*** e ***‘senh@f1ap’*** são o login e a senha, respectivamente. Feito isso, inicie novamente o *WildFly*.

Antes de executar essa classe, garanta que o servidor em que o projeto EBJ está instalado não esteja parado. Não é necessário estar em outro computador, pois pode testar do mesmo *host*, conforme o Código-fonte Classe Java que funciona como Cliente EJB, em que acessamos o IP do *localhost*.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Ao ser executada, ela deve se conectar com o servidor onde está o *“smarticities-ejb”*, e obter a “versão remota” do **EstabelecimentosService** numa instância na qual pode executar os métodos da interface **EstabelecimentosServiceRemote**.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

JBOSS. **DataSource configuration – WildFly 10**. Disponível em: <https://docs.jboss.org/author/display/WFLY10/DataSource+configuration?_sscc=t>. Acesso em: 17 jan. 2018.

JBOSS. **JBoss Tools – Documentation**. Disponível em: <https://tools.jboss.org/documentation/>. Acesso em: 17 jan. 2018.

ORACLE. **The Lifecycles of Enterprise Beans**. Disponível em: <https://docs.oracle.com/javaee/6/tutorial/doc/giplj.html>. Acesso em: 15 jan. 2018.

ORACLE. **Using the Timer Service**. Disponível em: <https://docs.oracle.com/javaee/6/tutorial/doc/bnboy.html>. Acesso em: 22 jan. 2018.

ORACLE. **What is a Session Bean?**. Disponível em: <https://docs.oracle.com/javaee/5/tutorial/doc/bnbly.html>. Acesso em: 17 jan. 2018.

WETHERBEE, Jonathan; KODALI, Raghu; RATHOD, Chirag; ZADROZNY, Peter. **Beginning EJB 3: Java EE**. 7. ed. New York: Apress, 2013.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com